

Autonomous Multi-Agent Systems for UAV Flight Safety Inspections using Large Language Models

Burak YORUK

Department of Computer Engineering

Ege University

İzmir, TURKEY

ORCID: 0000-0001-8980-191X

Aycan KAYA

Department of Computer Engineering

Ege University

İzmir, TURKEY

Duygu KAMALAK

Department of Computer Engineering

Ege University

İzmir, TURKEY

Abstract—Unmanned Aerial Vehicle (UAV) operations often rely on manual checklists and human intuition for flight safety, which introduces the risk of human error during complex or stressful missions. This paper introduces a fully autonomous Multi-Agent System (MAS) architecture designed to conduct real-time UAV flight safety inspections using Large Language Models (LLMs), specifically powered by Google’s Gemini 2.5 Flash model. By integrating live telemetry data—such as wind speed, battery health, and GPS satellite counts—with an interactive pre-flight checklist, the proposed system dynamically assesses operational risk.

Our architecture is based on the Foundation for Intelligent Physical Agents (FIPA) Agent Communication Language (ACL) standards. The system orchestrates five specialized agents within a Mission Supervisor state machine: Mission Supervisor, Safety Officer, Telemetry Analyst, Meteorology Agent, and Report Writer. The agents autonomously query environmental telemetry via simulated log readers and continuously evaluate whether conditions satisfy safety constraints, issuing preemptive NO-GO or Return-to-Home (RTH) commands even if the human operator manually verifies a checklist item. We demonstrate the practical feasibility of this approach through a lightweight web-based interface built with Flask and SQLite. The results indicate that delegating safety verification to an LLM-based MAS significantly reduces procedural oversights and increases mission reliability.

Index Terms—Multi-Agent Systems (MAS), Large Language Models (LLMs), Unmanned Aerial Vehicles (UAV), Flight Safety, FIPA-ACL, Autonomous Systems.

I. INTRODUCTION

As Unmanned Aerial Vehicles (UAVs) become increasingly integrated into industrial, agricultural, and emergency response applications, the complexity of managing their flight safety protocols has scaled proportionally. Traditional UAV operations mandate strict adherence to pre-flight, in-flight, and post-flight checklists to mitigate risks such as battery depletion, signal loss, or adverse weather conditions [1]. However, these checklists are predominantly static and rely heavily on the human operator’s manual input, situational awareness, and psychological state.

Recent advancements in Large Language Models (LLMs) offer unprecedented cognitive reasoning capabilities that can be embedded into autonomous systems [2]. While a single LLM agent might struggle to holistically monitor a complex cyber-physical system like a UAV, a Multi-Agent System (MAS) allows for the distribution of specialized tasks—such as risk analysis, maneuver advising, and logging—among several cooperating agents [3].

In this paper, we propose a novel framework that integrates an LLM-powered Multi-Agent System to oversee UAV flight safety. Unlike conventional systems where operators are the sole decision-makers, our architecture introduces agents that autonomously fetch live telemetry (e.g., weather data from Open-Meteo, simulated GPS, and battery metrics) and cross-validate the operator’s manual checklist inputs. If the external telemetry indicates unsafe conditions, the autonomous agents are empowered to override the operator and issue critical warnings or NO-GO directives.

The remainder of this paper is structured as follows. Section II details the architecture of our Multi-Agent System and its adherence to FIPA standards. Section III explains the integration of real-time telemetry into the agent decision loop. Section IV discusses the implementation results, and Section V concludes the study.

II. SYSTEM ARCHITECTURE

Our proposed system is structured around an interactive web interface powered by a Python Flask backend. The core intelligence is driven by an event-driven state machine (the *MissionSupervisor*), which seamlessly integrates simulated FIPA-ACL communications among multiple LLM-inspired agent roles. Furthermore, a local SQLite database (via *MissionStore*) persistently records JSON snapshots of the flight state.

A. Agent Roles and Responsibilities

To avoid creating a monolithic bottleneck, the cognitive workload is divided among five distinct agent roles operating within the supervisor:

- **Mission Supervisor (Coordinator):** Acts as the central hub and state machine controller. It receives

manual inputs from the operator’s checklist, manages flight states (e.g., *preflight*, *in_flight*), and delegates tasks to specialized agents.

- **Safety Officer:** Dedicated to evaluating risk factors and operator approvals. It handles critical Go/No-Go decisions and initiates Return-To-Home (RTH) procedures during low battery events.
- **Telemetry Analyst:** Continuously tracks route progression, GPS satellite lock, and positional deviations. It proposes holding positions or RTH when GPS accuracy drops.
- **Meteorology Agent:** Specialized in environmental safety. It parses wind and temperature data, proactively proposing route shortenings or immediate landings during high wind scenarios.
- **Report Writer:** Compiles session logs, communication histories, and safety overrides into a persistent historical record for post-flight analysis.

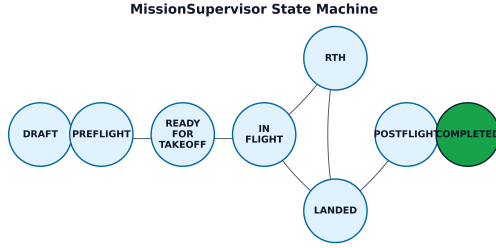


Figure 1. The MissionSupervisor State Machine, illustrating the progression from *draft* through *completed* states.

B. FIPA-ACL Communication Protocol

For the system to function deterministically, the agents must communicate using a standardized protocol. We adopted the Foundation for Intelligent Physical Agents (FIPA) Agent Communication Language (ACL) specifications [4].

As illustrated in Fig. 2, the communication flow follows a strict request-response paradigm. The backend maintains an internal `fipa_log` to record all transactions, ensuring that every multi-agent interaction is auditable and strictly typed by its performative:

Listing 1. FIPA-ACL Message Logging Implementation

```

def _fipa(self, sender: str, receiver: str,
    performative: str, content: str) -> None:
    self.fipa_log.append({
        "timestamp": utc_now(),
        "sender": sender,
        "receiver": receiver,
        "performative": performative,
        "content": content,
    })

```

For example, when telemetry indicates high wind, the Meteorology Agent issues a *PROPOSE* message. The

Mission Supervisor then evaluates this and may issue a *REQUEST* to the Safety Officer for an emergency landing decision, culminating in an *INFORM* message back to the operator interface.

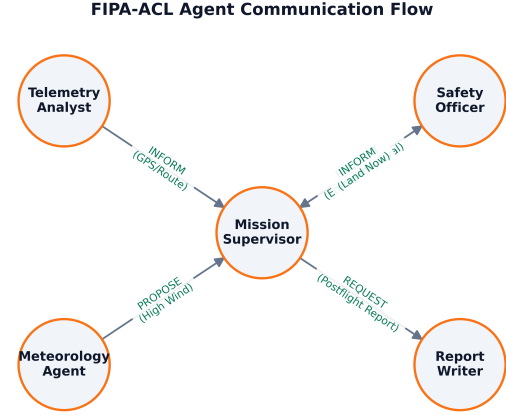


Figure 2. FIPA-ACL interactions among the UAV safety agents during a mission sequence.

III. TELEMETRY AND AUTONOMOUS OVERSIGHT

A critical vulnerability in traditional checklist applications is the assumption that the operator’s input is invariably correct. An operator might manually check off "Weather Conditions are Optimal" despite rapidly changing wind speeds.

To address this, our system introduces a `TelemetryLogReader` that streams simulated flight logs containing real-time values (battery percentage, GPS satellite lock count, wind speed, and heading). The `process_next_event()` loop feeds these data points to the respective agents for proactive checks.

The agents are explicitly instructed with critical safety boundaries:

- **Wind speed:** Must not exceed 12 m/s (NO-GO limit). Warnings are issued if wind reaches 9 m/s.
- **Battery level:** Must remain above 90% for a safe mission buffer; if it falls lower during *in_flight*, the Safety Officer warns the operator to calculate RTH reserves.
- **GPS lock:** Warnings are issued by the Telemetry Analyst if satellite lock drops to 15 or below.

If any of these conditions are violated, the specialized agents autonomously override the manual checklist progression. The *MissionSupervisor* evaluates these thresholds continuously as shown in Listing 2:

Listing 2. Autonomous Telemetry Risk Evaluation

```

def _risk_summary(self) -> dict:
    wind = self.telemetry_snapshot.get("wind")
    battery = self.telemetry_snapshot.get("battery")
    status = "GO"

```

```

notes = []

if wind is not None and wind >= 9:
    status = "WARNING"
    notes.append("Ruzgar sinira yaklasiyor.")
if battery is not None and battery < 90:
    status = "WARNING" if status == "GO" else
    status
    notes.append("Batarya gorev sonuna
    yaklasiyor.")

return {"status": status, "notes": notes}

```

They inject proactive alerts (e.g., "Wind early warning") into the operator's dashboard, creating a fail-safe mechanism where the system's logical deduction acts as a safety net against human error. This autonomous intervention logic is presented in Listing 3.

Listing 3. Proactive Autonomous Interventions

```

if wind is not None and wind >= 8:
    self._add_proactive(
        "wind-early-warning",
        "Meteorology Agent",
        "Ruzgar trendi yukseliyor",
        "Ruzgar limit yakini. Rota kisaltma veya
        erken inis icin hazirlik oneriliyor."
    ,
        severity="warning",
        actions=["Rota kisalt", "RTH hazirla"],
    )

```

Our implementation proves that multi-agent design patterns can transition from passive conversational bots to active, deterministic decision-makers in safety-critical domains. Future work will focus on integrating edge-based, locally hosted LLMs to provide real-time cognitive reasoning over the simulated FIPA-ACL logs, eliminating the latency and connectivity constraints associated with cloud-based API inference.

REFERENCES

- [1] R. Austin, *Unmanned Aircraft Systems: UAVS Design, Development and Deployment*. Wiley, 2010.
- [2] T. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877-1901, 2020.
- [3] M. Wooldridge, *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2009.
- [4] FIPA, "FIPA ACL Message Structure Specification," *Foundation for Intelligent Physical Agents*, SC00061G, 2002.

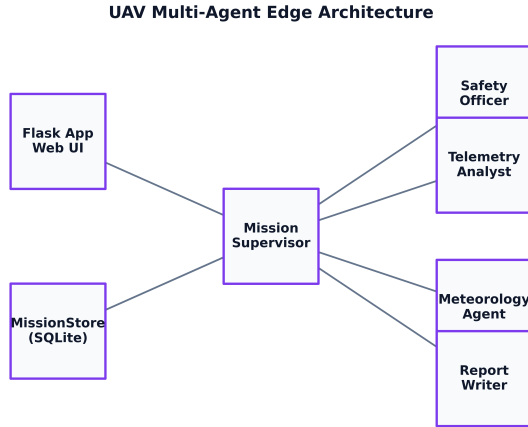


Figure 3. The complete UAV Multi-Agent Edge Architecture, showing the interaction between the Flask UI, SQLite storage, and the internal Agent State Machine.

IV. CONCLUSION AND FUTURE WORK

This paper presented an autonomous Multi-Agent System designed to enhance UAV flight safety through the integration of state-machine-driven agent roles and real-time telemetry. By utilizing FIPA-ACL standards, we established a robust communication pipeline among specialized agents capable of cross-validating human inputs against live environmental data.